VANI Quality Audit & Fix Loop

Demo video script, about 8 minutes

How to read this:
Yellow boxes tell you what to show on screen. Don't read them aloud.
Green "SAY" blocks are what you speak. Go slowly; the **numbers** matter more than the exact words.

Part 1	Intro	1 min
Part 2	Dashboard: Audit, Issues, Fix, Test, Live agents	4 min
Part 3	Setting the scene for the live test	30 s
Part 4	Live test on Sarvam (buyer lines)	2 min
Part 5	Close	20 s

Before you press record:

- Start the dashboard: `.venv/bin/streamlit run pipeline/dashboard.py`
- Open the Samvaad console with **VANI-before** and **VANI-after** in two browser tabs.
- Keep the "Part 4" page (the 8 buyer lines) open on a second screen or printed out.

The whole video follows **one problem** from start to end: **C3, the bot asks the same question again and again.**

Part 1: Intro (about 1 min)

SCREEN: Dashboard title, or a title slide: "VANI Quality Audit & Fix Loop".

SAY

VANI is IndiaMART's voice bot. When a buyer calls a seller who is busy, VANI picks up, understands what the buyer needs, and turns the call into a lead for sellers.

Today, quality is checked by hand. An auditor listens to a small sample of calls and fills a spreadsheet. So most failures are never heard. The reasons behind them are guessed. And fixes reach the bot's prompt slowly.

We built a loop that closes this gap. It audits every call, finds why calls fail, ranks those reasons, writes a fix to the prompt, and tests the fix, first offline and then live on Sarvam's voice agent platform.

Everything runs on Sarvam: Saaras for speech-to-text, Sarvam-105B for grading, root causes and fixes, and Sarvam Voice Agents for the live before-and-after test.

We'll follow one problem all the way through: the bot asking the same question again and again.

Part 2: Dashboard walkthrough (about 4 min)

Tab 1: Audit (about 1 min)

SCREEN: Audit tab. Point at the five numbers on top.

SAY

We transcribed and graded all **713** recordings, one hundred percent, not a sample.

Seventy-four calls were too short to have a conversation. Of the rest, **282 are Fatal** and **357 Non-Fatal**.

Every call is graded on the 17 parameters of IndiaMART's own call quality matrix. A grade only counts if it comes with a quote from the transcript and a timestamp, and the code checks that quote is really in the call.

SCREEN: Point at the agreement number.

SAY

To check the grader, we took calls from IndiaMART's real audit sheet and graded them blind. On the held-out half, we agree on Fatal versus not-Fatal **59%** of the time, and **67%** when weighted to the real mix of calls.

Most disagreements are calls the auditors failed because of the lead record in IndiaMART's system. That record isn't in the audio, so no grader listening to the call can see it.

SCREEN: Scroll to "Risky calls queue". Filter by flag: **loop**. Grade: **FATAL**. Open the **6th row**: call **209_90741505_958TFL4H5**.

SAY

547 calls are flagged for a human to look at: wrong requirement, frustration, loops, unclear audio, or the buyer asking for a person.

Here's one. The grader says the bot asked for the city and state **six times**, against a limit of three. Here's the exact line and the timestamp, 140 seconds in. An auditor can check this in seconds instead of listening to the whole call.

SCREEN (optional): Play the audio from about 2:20 for 10 seconds.

Tab 2: Issues (about 1 min)

SCREEN: Issues tab. Point at the bar chart, then the formula line.

SAY

Failed calls aren't a flat list. We cut them into batches of 40 and let the model find patterns in each batch. Patterns that keep coming back across batches are merged into root causes.

Each cause is ranked by priority, which is **frequency times severity**. Frequency is the share of all calls it affects. Severity counts a Fatal call as 3 and a Non-Fatal call as 1.

Each cause is also typed: a prompt problem, a speech-recognition problem, a knowledge gap, or a flow problem.

SCREEN: Point at the "Importance gate" box and the **updates_prompt** column.

SAY

Not every small problem should change the prompt. A cause can only update the prompt if it's a prompt problem, it affects at least 10 calls, and it shows up in at least three batches. Only **two causes** pass that gate.

SCREEN: Open **C3 "Bot loops on the same question"**.

SAY

Our cause is C3: the bot loops on the same question. It's in **62 calls**, almost 9% of everything, with 29 of them Fatal, and it showed up in **14 of the 15** batches.

Note the top cause, the bot mishearing quantities and specs. It's bigger, but it's a speech-recognition problem, so we don't pretend a prompt change will fix it.

Tab 3: Fix (about 45 s)

> **SCREEN:** Fix tab. Show the C3 card and its red/green diff.

> **SAY**
>
> For each gated cause, the system writes one small, specific change to the real VANI prompt, shown as a diff.
>
> It first finds the rule that should have prevented the problem, then puts the fix in the section that controls that behaviour. It can change behaviour only. If a change adds a price, a number or a link, it's rejected automatically.
>
> Before anything goes live, a reviewer checks it. We used Sarvam Cowork as that reviewer. It kept the loop fix but reworded it, and dropped the second fix, because the rule already exists in the prompt.

> **SCREEN:** Scroll down to the "Watchlist" and "Causes a prompt cannot fix" sections.

> **SAY**
>
> Smaller causes wait on a watchlist. Causes a prompt can't fix are listed with the reason.

Tab 4: Test (about 30 s)

> **SCREEN:** Test tab. Point at "Issue to tested fix: 23 min" and the timeline table.

SAY

First we tested offline. A simulated buyer replays the real buyer's lines from the failed calls, against the old prompt and the new one, and the same grader scores both. From finding the issue to a tested fix took **23 minutes**.

On these six calls, ask-limit loops dropped from **67% to 50%**, with no passing call broken. But six calls is a small sample, so we tested the fix live.

Tab 5: Live agents (about 1 min)

SCREEN: Live agents tab. Point at the two agent names, then the blue "only difference" box.

SAY

We built two real voice agents on Sarvam's platform, through the Voice Agents MCP. VANI-before runs IndiaMART's current prompt. VANI-after runs the same prompt plus two lines: a reworded question still counts as the same question, and the city is asked at most three times, after which the bot saves the lead and moves on.

SCREEN: Point at the eval table, then the "Fix iterations" table.

SAY

Sarvam's eval engine played a stubborn buyer who won't give a city, 20 times against each agent, and scored every conversation.

The current prompt passed **14 out of 20**. The fixed prompt passed **19 out of 20**. Asking for the city more than three times went from **5 calls to zero**. Losing the lead went from **4 calls to zero**. The other scenarios, a vague-quantity buyer and a cooperative buyer, still pass on both.

Our first attempt actually failed: we put the rule in a side field and it made no difference. Moving it into the section that controls location fixed it. That's why we test fixes instead of trusting them.

SCREEN: Scroll to "Same stubborn buyer, both agents". Point at the last bot line on each side.

SAY

Same buyer, both agents. On the left, the current bot keeps pushing for the city, then tells the buyer to call the seller back later. That lead is lost.

On the right, the fixed bot stops after its limit, notes the requirement, and promises seller details on WhatsApp. That lead is saved.

Part 3: Setting the scene (about 30 s)

SCREEN: Switch to the Samvaad console. Show the agent list with VANI-before and VANI-after.

SAY

Now let's see it live. These are the two agents on Sarvam's platform. Same voice, same languages, same product: a **10-ton soil compactor**, taken from a real call in our data where the bot asked for the city five times.

I'll play the same buyer to both agents. I'll confirm the product, give the quantity and specs, give my name, and then refuse to give a city. I'll just keep saying "it's nearby, just send me a seller".

Watch two things. First, how many times the bot asks for the city. The limit is three. Second, how the call ends: does the bot save the lead, or tell me to call back later?

First, the current prompt.

Part 4: Live test on Sarvam

SCREEN: Open **VANI-before** → test panel → **Chat** (most reliable) or **Web call**.
Say or type **one line per bot turn**. Wait for the bot to finish each time.

1. haan yes i need ten ton soil compactor only
2. one piece only, drum roller ten ton
3. ten ton only
4. my name is Sujit
5. paas mein hi hai, aap seller bhej do bas
6. arrey bataya na, yahin paas mein hi hai, seller bhej do
7. gaon mein rehte hain, bataya na, seller bhej do bas
8. yahin ke hain, aap bas seller ko details bhej do na

SAY

After lines 5 to 8, briefly: "That's the second time it asked for the city… third… fourth."

At the end, point at the bot's last line.

If it says **"same seller ko dobara call kar sakte hain"**: "The lead is lost."

If it says **"requirement note ho gayi hai… WhatsApp"**: "This time it held. The current prompt passes about 70% of the time, which is why we measured 20 runs, not one."

"Now the fixed prompt."

SCREEN: Open **VANI-after** and use the **same 8 lines**.

SAY

Point out: it asks for the city at most three times, it says the seller will confirm the location, and it closes with "requirement note ho gayi hai, seller details WhatsApp par". The lead is saved.

Recording tips

- Record 2 or 3 takes on VANI-before and keep the one where it loops or loses the lead. That happens in about 30% of runs.
- If no take fails, keep one take and cut back to the dashboard's side-by-side transcript.
- For a web call, speak clearly and wait for the bot to finish each turn. Background noise adds speech-recognition errors that have nothing to do with the fix.

Part 5: Close (about 20 s)

SCREEN: Back to the dashboard, Live agents tab.

SAY

So that's the full loop on one root cause. **713** calls audited. Looping found as a ranked, prompt-fixable cause. A two-line, reviewed prompt change. And live tests on Sarvam: from **70% to 95%** passing, and lost leads from **4 to zero**.

It's a reviewable fix, not an automatic change to production. The same pipeline runs again on the next batch of calls.